Lab Assignment –7 (Transactions & Concurrency)

Department: CSE

Subject: DIS Lab
Code: 22CPP217

Objective: Understand ACID properties through practical examples, implement transactions using COMMIT, ROLLBACK, and SAVEPOINT, observe concurrency issues such as lost updates and dirty reads, and relate transaction control to real-world multi-user database systems.

Pre-Lab Setup: Run this block to initialize a Banking environment to simulate financial transactions.

```sql
-- 0. Reset Environment
SET SQL_SAFE_UPDATES = 0;
DROP TABLE IF EXISTS Accounts;

-- 1. Create Schema
CREATE TABLE Accounts (
    Account_ID INT PRIMARY KEY,
    Account_Name VARCHAR(50),
    Balance DECIMAL(10, 2) CHECK (Balance >= 0)
);

-- 2. Insert Sample Data
INSERT INTO Accounts VALUES
(101, 'Alice', 5000.00),
(102, 'Bob', 3000.00),
(103, 'Charlie', 1000.00);
```

Q 1: The Classic Transfer (COMMIT & ROLLBACK) The bank requires that transferring money between accounts must be strictly processed as a single unit of work (Atomicity).

- Start a new transaction.
- Transfer ₹1000 from Alice (Account 101) to Bob (Account 102). Deduct from Alice and add to Bob.
- **COMMIT** the transaction and verify both balances.
- Start another transaction to transfer ₹6000 from Bob to Charlie. Deduct ₹6000 from Bob (this will violate the CHECK constraint).
- **ROLLBACK** the transaction. Verify that Bob's balance was not partially updated and remains intact.

Q 2: Multi-Step Processing (SAVEPOINT) Complex operations sometimes require partial rollbacks without abandoning the entire transaction.

- Start a new transaction for Charlie (Account 103) to book a trip.
- Deduct ₹400 for a flight ticket.
- Create a **SAVEPOINT** named `Flight_Booked`.
- Attempt to deduct ₹800 for a hotel booking (this exceeds his remaining ₹600 balance).
- Instead of rolling back everything, **ROLLBACK TO** `Flight_Booked`.
- **COMMIT** the transaction. Check Charlie's balance to ensure the flight was booked (Balance = ₹600) but the hotel was not.

Q 3: Concurrency Issues (Dirty Reads & Isolation Levels) Transactions occurring simultaneously can interfere with each other. *(Note: For this exercise, open TWO separate MySQL sessions/tabs).*

- **Session A:** Set the transaction isolation level to `READ UNCOMMITTED`. Start a transaction and update Alice's balance to ₹10000, but **DO NOT COMMIT** yet.
- **Session B:** Set the transaction isolation level to `READ UNCOMMITTED`. Query Alice's balance. You will see ₹10000 (a Dirty Read).
- **Session A: ROLLBACK** the transaction.
- **Session B:** Query Alice's balance again. Notice how Session B previously read invalid data that never truly existed in the committed database.
- Reset your isolation level to `READ COMMITTED` or `REPEATABLE READ` and try the experiment again to see how the database prevents the dirty read.

Q 4: The Race Condition (Lost Updates & Locking) In a multi-user environment, two tellers might try to update the same account at the exact same time. If not handled correctly, one teller's update might overwrite the other's, resulting in a "Lost Update." *(Note: Use TWO separate MySQL sessions for this).*

- **Session A:** Start a transaction. Check Bob's balance (Account 102).
- **Session B:** Start a transaction. Check Bob's balance at the exact same time.

- **Session A:** Add ₹500 to Bob's balance (pretend you are calculating this manually based on the read) and execute the `UPDATE`.
- **Session B:** Deduct ₹200 from Bob's balance (also calculated manually based on the initial read) and execute the `UPDATE`.
- **Both Sessions: COMMIT**. Notice how the final balance only reflects the last update, completely losing the other transaction's data.
- **The Fix:** Reset the balance. Repeat the experiment, but this time use `SELECT ... FOR UPDATE` in Session A. Observe how Session B is now forced to wait until Session A commits before it can even read the row.

SQL Reference:

```sql
-- 1. TRANSACTION STRUCTURE
START TRANSACTION;
-- Write your UPDATE / INSERT / DELETE statements here
COMMIT; -- to permanently save changes
-- OR
ROLLBACK; -- to undo changes


-- 2. SAVEPOINT STRUCTURE
START TRANSACTION;
-- SQL statements...
SAVEPOINT Savepoint_Name;
-- More SQL statements...
ROLLBACK TO Savepoint_Name; -- Undoes only the statements after the savepoint
COMMIT;


-- 3. SETTING ISOLATION LEVELS
SET SESSION TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
-- Options: READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE


-- 4. ROW-LEVEL LOCKING (Preventing Lost Updates)
START TRANSACTION;
-- Locks the selected rows until the transaction is committed or rolled back
SELECT Balance FROM Accounts WHERE Account_ID = 102 FOR UPDATE;
-- Perform your calculations and updates here
UPDATE Accounts SET Balance = 3500 WHERE Account_ID = 102;
COMMIT;
```